MANUAL TÉCNICO ESPECIALIZADO • VOLUME 1

Dominando o SQL: Fundamentos e CRUD

Da modelagem básica de tabelas à extração analítica com SELECT e resolução prática de cenários reais.

1. Arquitetura Relacional & Modelo de Dados

A linguagem **SQL** (*Structured Query Language*) é a interface padrão internacional para armazenamento, manipulação e extração de informações em Sistemas Gerenciadores de Bancos de Dados Relacionais (SGBDs), como PostgreSQL, MySQL, MariaDB e SQLite. Em sua essência, tabelas estruturam registros (linhas) indexados por atributos rígidos (colunas).

> **Estrutura de Apoio:** Para todos os exemplos e exercícios práticos deste material, utilizamos o catálogo de produtos comerciais definido abaixo via DDL (*Data Definition Language*):

```
CREATE TABLE produtos (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    categoria VARCHAR(50) NOT NULL,
    preco DECIMAL(10, 2) NOT NULL,
    estoque INT DEFAULT 0,
    ativo BOOLEAN DEFAULT TRUE,
    data_cadastro DATE DEFAULT CURRENT_DATE
);
```

2. Visão Panorâmica do Ciclo CRUD

As quatro operações fundamentais em persistência de dados formam o acrônimo **CRUD**. Neste primeiro volume, exploramos a manipulação de dados em uma única entidade antes de avançarmos para junções relacionais no próximo volume.

Operação	Comando SQL	Função Principal	Observação Técnica
Create	`INSERT INTO`	Persistência de novos registros	Pode omitir colunas que contenham cláusula `DEFAULT` ou `SERIAL`.
Read	`SELECT`	Recuperação e projeção de dados	Apresenta granularidade de filtragem, cálculos, ordenação e agrupamentos.
Update	`UPDATE`	Mutação de valores existentes	Exige rigor na cláusula `WHERE` para evitar alterações em massa indesejadas.
Delete	`DELETE FROM`	Exclusão física de registros	Diferencia-se do TRUNCATE por permitir filtros e registrar logs transacionais.

3. Operações de Modificação: INSERT, UPDATE e DELETE

3.1 Inserção de Dados (Create • INSERT)

A gravação aceita tanto chamadas pontuais quanto inserções eficientes em lote (*batch insert*), minimizando acessos ao disco:

```sql
-- 1. Inserção pontual completa
INSERT INTO produtos (nome, categoria, preco, estoque, ativo, data_cadastro)
VALUES ('Teclado Mecânico RGB', 'Periféricos', 289.90, 45, TRUE, '2026-01-15');

-- 2. Inserção de lote com omissão de colunas com DEFAULT
INSERT INTO produtos (nome, categoria, preco, estoque) VALUES
('Monitor 27" 144Hz', 'Monitores', 1450.00, 15),
('Cadeira Ergonômica', 'Móveis', 890.00, 12),
('Fone Bluetooth Over-Ear', 'Áudio', 350.00, 0),
('Webcam Full HD 1080p', 'Periféricos', 210.00, 25),
('Microfone Condensador USB', 'Áudio', 310.00, 18),
('Cabo HDMI 2.1 2m', 'Cabos', 45.00, 150);
```

3.2 Atualização de Registros (Update • UPDATE)

Modifica colunas específicas usando expressões matemáticas ou regras de integridade de negócio:

```sql
-- 1. Alteração pontual com base na Chave Primária (PK)
UPDATE produtos SET preco = 269.90, estoque = 40 WHERE id = 1;

-- 2. Reajuste inflacionário de 10% apenas para categoria ativa
UPDATE produtos
SET preco = ROUND(preco * 1.10, 2)
WHERE categoria = 'Periféricos' AND ativo = TRUE;

-- 3. Inativação automática baseada em regra de estoque
UPDATE produtos SET ativo = FALSE WHERE estoque = 0;
```

> **Regra Crítica de Segurança:** As instruções UPDATE e DELETE afetam todas as linhas caso a cláusula WHERE seja omitida. Em ambientes de produção, sempre execute previamente um SELECT ... WHERE equivalente para auditar os registros atingidos.

3.3 Remoção de Registros (Delete • DELETE)

```sql
-- Remoção cirúrgica com múltiplos predicados
DELETE FROM produtos WHERE ativo = FALSE AND estoque = 0;
```

4. O Motor de Consultas: SELECT Essencial & Filtros

O SELECT viabiliza leitura seletiva, cálculos derivados, ordenação e regras lógicas de filtragem sem alterar o estado original das tabelas.

4.1 Projeção, Renomeação (Aliases) e Cálculos Derivados

```sql
-- Consulta limpa com cálculo patrimonial e simulação de pagamento à vista (-15%)
SELECT
    nome AS produto,
    preco AS preco_cheio,
    estoque,
    (preco * estoque) AS valor_total_em_estoque,
    ROUND(preco * 0.85, 2) AS preco_pix_desconto
FROM produtos;
```

4.2 Lógica Booleana e Operadores de Comparação

```sql
-- Precedência explícita com parênteses em expressões combinadas
SELECT id, nome, categoria, preco
FROM produtos
WHERE (categoria = 'Áudio' OR categoria = 'Monitores')
  AND preco >= 300.00
  AND ativo = TRUE;
```

4.3 Filtros Especiais: BETWEEN, IN, LIKE e Tratamento de Nulos

```sql
-- 1. Faixas de valores contínuos (inclusivo)
SELECT nome, preco FROM produtos WHERE preco BETWEEN 100.00 AND 450.00;

-- 2. Pertencimento a conjuntos discretos
SELECT nome, categoria FROM produtos WHERE categoria IN ('Áudio', 'Móveis');

-- 3. Reconhecimento de padrões em strings (%: qualquer caractere)
SELECT nome, preco FROM produtos WHERE nome LIKE '%Sem Fio%';

-- 4. Verificação de nulidade (nunca use = NULL)
SELECT nome FROM produtos WHERE estoque IS NULL OR estoque = 0;
```

> **Padrão ANSI para NULL:** Em SQL, `NULL` representa ausência de valor ou dado desconhecido. Por esse motivo, comparações como `coluna = NULL` ou `coluna != NULL` retornam *UNKNOWN* em vez de verdadeiro. Use sempre `IS NULL` ou `IS NOT NULL`.

5. Agrupamentos, Métricas e Ordem de Execução

5.1 Ordenação e Paginação Determinística

```sql
-- Paginação técnica (Página 2 exibindo 3 registros por requisição)
SELECT id, nome, preco
FROM produtos
WHERE ativo = TRUE
ORDER BY preco DESC, id ASC
LIMIT 3 OFFSET 3;
```

5.2 Agregações e Filtros de Grupo: GROUP BY & HAVING

Enquanto o `WHERE` filtra linhas individuais antes de qualquer processamento, o `HAVING` avalia condições sobre grupos consolidados:

```sql
SELECT
    categoria,
    COUNT(*) AS total_produtos,
    SUM(estoque) AS pecas_totais,
    ROUND(AVG(preco), 2) AS preco_medio
FROM produtos
WHERE ativo = TRUE
GROUP BY categoria
HAVING AVG(preco) > 200.00
ORDER BY preco_medio DESC;
```

6. A Ordem Lógica de Processamento do Banco

O SGBD não processa o código SQL na ordem em que ele é escrito. A compreensão do fluxo interno é indispensável para evitar o uso incorreto de aliases e índices:

Ordem	Cláusula	Operação Realizada pelo Motor
1º	`FROM`	Carrega as tabelas fontes e referências da memória ou disco.
2º	`WHERE`	Aplica filtros linha a linha (descarta registros inválidos).
3º	`GROUP BY`	Organiza as linhas sobreviventes em buckets/categorias.
4º	`HAVING`	Descarta buckets que não satisfazem o critério de agregação.
5º	`SELECT`	Computa projeções, funções matemáticas e define aliases.
6º	`ORDER BY`	Ordena o conjunto final baseado em critérios de classificação.
7º	`LIMIT / OFFSET`	Fatia a janela de visualização entregue à aplicação cliente.

7. Caderno Prático: Exercícios de Fixação (1 a 5)

Pratique as instruções essenciais de manipulação e consulta utilizando a base de dados comercial `produtos`.

Exercício 01 • Inserção Pontual (Create) BÁSICO

Enunciado: Cadastre um novo produto com o nome `'Mousepad Gamer Extra Grande'`, categoria `'Acessórios'`, valor unitário de `89.90`, com 60 unidades em estoque e status TRUE.

Exercício 02 • Modificação Condicional (Update) INTERMEDIÁRIO

Enunciado: Conceda um desconto de 15% sobre o preço atual de todos os itens cadastrados sob a categoria `'Áudio'` cujo preço original seja superior a `300.00`. Arredonde o resultado em 2 casas decimais.

Exercício 03 • Exclusão Controlada (Delete) INTERMEDIÁRIO

Enunciado: Remova do catálogo os registros que estejam inativos (`ativo = FALSE`), que pertençam à categoria `'Acessórios'` e cujo estoque esteja zerado.

Exercício 04 • Projeção e Ordenação (Read) BÁSICO

Enunciado: Recupere as colunas `nome`, `preco` e `estoque` de todos os itens ativos, ordenados do produto de maior valor para o de menor valor comercial.

Exercício 05 • Filtragem por Intervalo e Negação BÁSICO

Enunciado: Selecione os campos `nome`, `preco` e `categoria` para os produtos cujo preço situe-se entre `150.00` e `500.00` (inclusivo), excluindo a categoria `'Móveis'`.

7. Caderno Prático: Exercícios de Fixação (6 a 10)

Consolidação de consultas analíticas, limites, busca textual, funções agregadas e segmentação de grupos.

Exercício 06 • Reconhecimento de Padrão Textual INTERMEDIÁRIO

Enunciado: Selecione todas as colunas dos produtos cujo nome contenha o termo `'Monitor'` ou o termo `'USB'` em qualquer posição textual (sem considerar distinção de chave primária).

Exercício 07 • Consolidação Estatística Global BÁSICO

Enunciado: Escreva uma consulta que apresente em uma única linha o número total de itens cadastrados, a soma do estoque físico total e a média geral de preço com 2 casas decimais.

Exercício 08 • Agrupamento por Categoria (GROUP BY) INTERMEDIÁRIO

Enunciado: Monte um demonstrativo contendo o nome de cada categoria, o total de itens ativos existentes nela e o maior preço unitário verificado em cada uma.

Exercício 09 • Filtragem sobre Grupos (HAVING) AVANÇADO

Enunciado: Exiba o nome da categoria e o somatório de unidades em estoque exclusivamente para os grupos que acumularem mais de 50 itens físicos armazenados.

Exercício 10 • Relatório Paginado com Cálculo Comercial AVANÇADO

Enunciado: Projete a primeira página de um catálogo eletrônico contendo apenas itens com estoque disponível (`estoque > 0`). Exiba `id`, `nome`, `preco` e o valor de cada prestação em 10 parcelas sem juros (`preco / 10`), ordenado por preço crescente e limitado a 4 itens.

8. Gabarito Comentado: Exercícios 01 a 05

Solução 01 • Inserção de Registro

```sql
INSERT INTO produtos (nome, categoria, preco, estoque, ativo)
VALUES ('Mousepad Gamer Extra Grande', 'Acessórios', 89.90, 60, TRUE);
```

Explicação: Não é necessário declarar o campo `data_cadastro` porque ele assume o valor corrente do sistema via `DEFAULT CURRENT_DATE`. A chave `id` é gerada automaticamente pelo tipo `SERIAL`.

Solução 02 • Atualização com Fator Residual

```sql
UPDATE produtos
SET preco = ROUND(preco * 0.85, 2)
WHERE categoria = 'Áudio' AND preco > 300.00;
```

Explicação: Multiplicar por 0.85 equivale a reter 85% do valor original (15% de abatimento). O uso da função `ROUND(..., 2)` evita dízimas fracionárias e respeita a precisão do tipo `DECIMAL(10,2)`.

Solução 03 • Remoção Restrita

```sql
DELETE FROM produtos
WHERE ativo = FALSE
  AND categoria = 'Acessórios'
  AND estoque = 0;
```

Explicação: O operador lógico AND assegura que o registro só será excluído se todas as três condições forem verdadeiras simultaneamente, prevenindo a remoção de itens com saldo em estoque.

Solução 04 • Projeção com Ordenação Decrescente

```sql
SELECT nome, preco, estoque
FROM produtos
WHERE ativo = TRUE
ORDER BY preco DESC;
```

Explicação: O modificador DESC na cláusula `ORDER BY` inverte a classificação padrão (que é ascendente, ASC), listando os produtos mais caros no topo do resultado.

Solução 05 • Intervalo e Diferença Relacional

```sql
SELECT nome, preco, categoria
FROM produtos
WHERE preco BETWEEN 150.00 AND 500.00
  AND categoria <> 'Móveis';
```

Explicação: O predicado `BETWEEN x AND y` é inclusivo nas duas pontas. O operador <> (ou `!=`) filtra valores distintos do literal fornecido.

8. Gabarito Comentado: Exercícios 06 a 10

Solução 06 • Busca Textual com Curinga

```sql
SELECT * FROM produtos
WHERE nome LIKE '%Monitor%'
   OR nome LIKE '%USB%';
```

Explicação: O caractere curinga % indica correspondência com qualquer sequência de caracteres à esquerda ou à direita do termo. O operador OR garante o retorno se ao menos um dos critérios for atendido.

Solução 07 • Funções Agregadas Escalares

```sql
SELECT
    COUNT(*) AS total_cadastrado,
    SUM(estoque) AS volume_estoque,
    ROUND(AVG(preco), 2) AS preco_medio
FROM produtos;
```

Explicação: Quando funções agregadas são usadas sem GROUP BY, a tabela inteira é tratada como uma partição única, condensando todas as linhas em um único registro resumido.

Solução 08 • Agrupamento por Categoria

```sql
SELECT
    categoria,
    COUNT(*) AS total_ativos,
    MAX(preco) AS maior_preco
FROM produtos
WHERE ativo = TRUE
GROUP BY categoria;
```

Explicação: O filtro WHERE é avaliado antes do particionamento. Em seguida, o GROUP BY distribui as linhas ativas entre as categorias e calcula os valores agregados para cada bloco.

Solução 09 • Filtragem de Agregações com HAVING

```sql
SELECT categoria, SUM(estoque) AS estoque_total
FROM produtos
GROUP BY categoria
HAVING SUM(estoque) > 50;
```

Explicação: O comando WHERE não suporta agregações, pois opera linha a linha. Para filtrar sobre a soma dos grupos gerados, o uso da cláusula HAVING é obrigatório.

Solução 10 • Paginação com Coluna Calculada

```sql
SELECT
    id, nome, preco,
    ROUND(preco / 10.0, 2) AS preco_parcelado_10x
FROM produtos
WHERE estoque > 0
ORDER BY preco ASC
LIMIT 4 OFFSET 0;
```

Explicação: A divisão direta `preco / 10.0` calcula o valor da parcela na projeção. O comando `ORDER BY ... LIMIT 4` implementa uma paginação determinística e controlada para a aplicação.